

SEN800 Lab 2 Report

Symmetric, Asymmetric, and Homomorphic Encryption

Student Name: Ma Toan Bach
Student Number: 112708227

June 10, 2026

Part 1: Symmetric Encryption with Python and OpenSSL

Exercise 1.1: Python Fernet Implementation

Step 1: Install the required Python library

```
~/Documents/last-term/SEN800/lab2> python3 -m pip install cryptography
Requirement already satisfied: cryptography in /opt/anaconda3/lib/python3.12/site-packages (43.0.0)
Requirement already satisfied: cffi>=1.12 in /opt/anaconda3/lib/python3.12/site-packages (from cryptography) (1.17.1)
Requirement already satisfied: pycparser in /opt/anaconda3/lib/python3.12/site-packages (from cffi>=1.12->cryptography) (2.21)
```

Step 2 and Step 3: Run the script multiple times

```
~/Documents/last-term/SEN800/lab2> python3 symmetric_fernet.py
Your secret key is: TwRYkr1UXR5_vqRLhMfuGxcDVg0s2u_qtRCiYNqs4ts=

Encrypted token:
gAAAAABqKfhfn1LIVwdDFRn6b1wxmUIL9bmQxnRBk_q4upS7kPVV-
FUttr2XjkN0t4aBDykzbyECUWb1TjW6BbiherPgYk1RFly3YUot52kLTA70q2Vnz-74mU90fjAt-4
LnSsoMHghi

Decrypted message: Hello! This is my secret message for the lab.

~/Documents/last-term/SEN800/lab2> python3 symmetric_fernet.py
Your secret key is: PKYgmZMelnFO_I2iTXohaqB97Pvnrz7g4ySdulLaGw4=

Encrypted token:
gAAAAABqKfh8Cp6We-xCi_qJEVvPdOaiqTkcnr6r47_q0Bj9Kgp-
zJFoeGDaELCFb721B35D3oFqyn1bm9vmoMBmk6SP9QGFP5TSyeBALmE3eahaAA007ow1lbzsJU8mVR
-BUXuxE-8i
```

```
Decrypted message: Hello! This is my secret message for the lab.
```

Observation

The plaintext stayed the same, but the encrypted token changed on each run. Both ciphertexts still decrypted back to the original message.

Scenario Question 1.1 Answer

Yes, the token changes each time the script is run, even when the `original_message` is identical. This is desirable because it prevents an attacker from learning that the same plaintext was encrypted more than once. Fernet uses fresh randomness for every encryption, including a new initialization vector within its AES-CBC-based construction, so the ciphertext is non-deterministic. That randomness hides patterns while still allowing correct decryption with the same secret key.

Exercise 1.2: Command-Line Encryption with OpenSSL

Step 1: Create a plain text file

```
~/Documents/last-term/SEN800/lab2> echo "This is a highly confidential document."
> secret.txt
```

Step 2: Encrypt the file using AES-256-CBC

```
~/Documents/last-term/SEN800/lab2> openssl enc -aes-256-cbc -pbkdf2 -in secret.
txt -out secret.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
```

Step 3: Decrypt the file and verify it works

```
~/Documents/last-term/SEN800/lab2> openssl enc -d -aes-256-cbc -pbkdf2 -in secret
.enc -out recovered.txt
enter AES-256-CBC decryption password:

~/Documents/last-term/SEN800/lab2> cat recovered.txt
This is a highly confidential document.
```

Step 4: Tamper with the encrypted file and decrypt it again

```
~/Documents/last-term/SEN800/lab2> openssl enc -d -aes-256-cbc -pbkdf2 -in secret
.enc -out tampered.txt
enter AES-256-CBC decryption password:

~/Documents/last-term/SEN800/lab2> cat tampered.txt
[binary garbage at start of line] confidential document.
```

Supporting Values

```
~/Documents/last-term/SEN800/lab2> openssl base64 -in secret.enc
U2FsdGVkX1/SKZM0vaaQky9wqzBvs98FruZ8Q3Gtpn/gWii0CW0wiBBazdp3FW0y
bVEznAhp+cvv3vANC5gULQo=

~/Documents/last-term/SEN800/lab2> shasum -a 256 secret.txt secret.enc recovered.
txt
cf4dd55320525a1b463abfb01b17dead2bb1f2a98db47e9d65c7bf31d4e5eeea  secret.txt
4112fa7044c4d936daff744e3c1d2bf78daff67032ec14f4491fec7433736888  secret.enc
ac2f2837ef23e6bbbd952f8bec39744e47762ae2125864e44c3d5b89f80ecb6a  recovered.txt
```

Observation

The decrypted plaintext matched the original file before tampering. After a one-byte modification to the ciphertext, decryption still returned output, but the plaintext was corrupted.

Scenario Question 1.2 Answer

When I decrypted the tampered ciphertext, the output was corrupted instead of matching the original file. This shows that CBC mode provides confidentiality, but not integrity or authenticity by itself. An attacker can alter ciphertext and the receiver may only discover the problem after decryption, or may not notice immediately. Authenticated encryption modes such as GCM include an authentication tag, so any tampering causes decryption to fail rather than return damaged plaintext. This experiment demonstrates why modern systems prefer authenticated encryption instead of confidentiality-only modes.

Part 2: Asymmetric Encryption with GPG

Exercise 2.1: Key Generation and File Encryption

Step 1: Generate a key pair

The interactive key generation step was completed earlier in the lab. The command and the resulting key fingerprint are shown below as evidence of the generated key currently stored in GPG.

```
~/Documents/last-term/SEN800/lab2> gpg --gen-key
[interactive prompts completed earlier in the lab]

~/Documents/last-term/SEN800/lab2> gpg --list-keys --fingerprint "toan <
toan@gmail.com>"
pub  ed25519 2026-06-10 [SC] [expires: 2029-06-09]
      8C09 D0F3 36E0 DB0E AD8C D00F B93C 2070 35E4 F54D
uid          [ultimate] toan <toan@gmail.com>
sub  cv25519 2026-06-10 [E] [expires: 2029-06-09]
      BF09 B154 1DE1 DD62 BE55 D47A 2333 679E 9176 AA4F
```

Step 2: Export the public key

```
~/Documents/last-term/SEN800/lab2> gpg --export -a "toan <toan@gmail.com>" >
  my_public_key.asc
~/Documents/last-term/SEN800/lab2> shasum -a 256 my_public_key.asc
8611442b97fc25830ffb25d0b5c9856546a00bec1edddebaa758e6272958c47a  my_public_key.
  asc

~/Documents/last-term/SEN800/lab2> cat my_public_key.asc
-----BEGIN PGP PUBLIC KEY BLOCK-----

mDMEainy6hYJKwYBBAHArW8BAQdAgaSrT4buCPDaSvkqBdF7+oyRPubhAYfjTCQr
ONxA2Vi0FXRvYW4gPHRvYW5AZ21haWwY29tPoi1BBMWCgBdFiEEjAnQ8zbg2w6t
jNAPuTwgcDXk9U0FAmop8uobFIAAAAAABAAObWFudTIsMi41KzEuMTIsMCwzAhsD
BQkFo5qABQsJCAcCAiICBhUKCQgLAGQWAgMBAh4HAheAAAoJELk8IHA15PVN8hQA
/jb5a2KSf0d6f5fqgWp4cGMwwpiQbXa790WyDnfdSeiXAQCysGzyEfz4Z+OrDuof
00dHrbwoE5uHjeQIr1G+YBilDrg4BGop8uoSCisGAQQB1lUBBQEBOBT0Cybjzn4
5GhXKKHigIXsbhm6R6z0syFrOnABm8QEFgMBCAeImgQYFgoAQhYhBIwJOPM24Ns0
rYzQD7k8IHA15PVNBQJqKfLqGxSAAAAAAQADm1hbnUyLDIuNSsxLjEyLDAsMwIb
DAUJBaOagAAKCRc5PCBwNeT1TZfBAQCvgkxJBjJlgnQnGTsLNx96f9FbZ+RXjk22N
dI7luqxV2QD/ctba58ncQQypBcvkB60z6M6qfHk4ku7MQJuItIKPeA4=
=h6Dt
-----END PGP PUBLIC KEY BLOCK-----
```

Step 3: Encrypt a message for yourself

```
~/Documents/last-term/SEN800/lab2> echo "Only the holder of the private key can
  read this." > private_msg.txt
~/Documents/last-term/SEN800/lab2> gpg -e -r "toan <toan@gmail.com>" -o
  private_msg_step3.gpg private_msg.txt

~/Documents/last-term/SEN800/lab2> shasum -a 256 private_msg.txt private_msg.txt.
  gpg
5623dabd55db145fe4b4c4c25a88dd2df8cd53b0b199bcf0d35a5afba992fd7f  private_msg.txt
3d2761f8fe2d1765af3b706470495d5505582fce0521575ef6b01e3ff5fe7233  private_msg.txt
  .gpg

~/Documents/last-term/SEN800/lab2> openssl base64 -in private_msg.txt.gpg
hF4DIzNnnpF2qk8SAQdABhC9SptARGdKdZPrItNTFkiKOG9QeWhfw9dT50wgFgw
OZ4q9xrRO51N8+PDqa+F0oBstpQMF7iE1VKP6Wdm7w4RJCJ55yISju6f05JjPN9D
1H0BCQIQ5FD0mr+pngBqb2C1xZic7tijQu1lE8YxEr4K5BVKK0wFlN61Gzh7tkJ
KfDtS8j4m9EloNvnHlkmZGLIi4m1ElexXPIqbWj+1MODqtX8eBd038qQ+sNGYgX0
JjrpfYv1Vf6aDsDqdGARvLTsGz1Q4DFXRvwSTwgcQ==
```

Step 4: Decrypt the message using the private key

```
~/Documents/last-term/SEN800/lab2> gpg -d private_msg.txt.gpg
gpg: encrypted with cv25519 key, ID 2333679E9176AA4F, created 2026-06-10
"toan <toan@gmail.com>"
```

```
[GPG then opens an interactive passphrase prompt for the private key in a real terminal.]
```

Observation

The key fingerprint confirms the generated key pair, the exported public key block is available for sharing, and the encrypted file has a different hash and byte content from the plaintext. Decrypting the message requires the private key and its passphrase.

Exercise 2.2: Digital Signatures

Step 1: Create a document and sign it

The signed file `contract.txt.gpg` was created earlier in the lab. The original command flow and the existing files below show the signed artifact and the recovered plaintext.

```
~/Documents/last-term/SEN800/lab2> echo "I agree to pay $100." > contract.txt
~/Documents/last-term/SEN800/lab2> gpg --sign contract.txt
[interactive signing completed earlier in the lab]

~/Documents/last-term/SEN800/lab2> shasum -a 256 contract.txt contract.txt.gpg
    verified_contract.txt
46deda09db99a8a06ce48ad4b6d4055f5492b4cea8416abddee587acc8359509  contract.txt
e5a17175442efa629d2d66d43ae54685b5c5ae39141d8fa6142bc98f269d761f  contract.txt.
    gpg
46deda09db99a8a06ce48ad4b6d4055f5492b4cea8416abddee587acc8359509
    verified_contract.txt
```

Step 2: Verify the signature

```
~/Documents/last-term/SEN800/lab2> gpg --verify contract.txt.gpg
gpg: Signature made Wed 10 Jun 19:35:11 2026 EDT
gpg:                using EDDSA key 8C09D0F336E0DB0EAD8CD00FB93C207035E4F54D
gpg: Good signature from "toan <toan@gmail.com>" [ultimate]
```

Step 3: Extract the original document

```
~/Documents/last-term/SEN800/lab2> gpg --decrypt contract.txt.gpg >
    verified_contract.txt
gpg: Signature made Wed 10 Jun 19:35:11 2026 EDT
gpg:                using EDDSA key 8C09D0F336E0DB0EAD8CD00FB93C207035E4F54D
gpg: Good signature from "toan <toan@gmail.com>" [ultimate]

~/Documents/last-term/SEN800/lab2> cat verified_contract.txt
I agree to pay $100.
```

Supporting Values

```

~/Documents/last-term/SEN800/lab2> gpg --list-packets contract.txt.gpg
# off=0 ctb=a3 tag=8 hlen=1 plen=0 indeterminate
:compressed packet: algo=1
# off=2 ctb=90 tag=4 hlen=2 plen=13
:onepass_sig packet: keyid B93C207035E4F54D
      version 3, sigclass 0x00, digest 10, pubkey 22, last=1
# off=17 ctb=ac tag=11 hlen=2 plen=39
:literal data packet:
      mode b (62), created 1781134511, name="contract.txt",
      raw data: 21 bytes
# off=58 ctb=88 tag=2 hlen=2 plen=145
:signature packet: algo 22, keyid B93C207035E4F54D
      version 4, created 1781134511, md5len 0, sigclass 0x00
      digest algo 10, begin of digest 78 a9
      hashed subpkt 33 len 21 (issuer fpr v4 8
C09D0F336E0DB0EAD8CD00FB93C207035E4F54D)
      hashed subpkt 2 len 4 (sig created 2026-06-10)
      hashed subpkt 20 len 26 (notation: manu=2,2.5+1.12,0,3)
      subpkt 16 len 8 (issuer key ID B93C207035E4F54D)
      data: [256 bits]
      data: [254 bits]

~/Documents/last-term/SEN800/lab2> openssl base64 -in contract.txt.gpg
owGbwMvMwCW200ahwPTJv1/GNepJPMn5eSVFickleiUVJVmaX9Z7KiSmF6WmKpTk
KxQkViqoGB0Y6HF1TGRhEONisBRTZOnhvPDZ7MFtvrU9F/hhBrEygXRKizQwAAEL
A19uYl6pkY6Rnqm2oZ6hkY6BjjEDF6cATHXFSkaGj0rmvC8qI+/frjwaesVC6uja
g5aZ9qeF7ANidK2LDu1+zPBP8Wqfwfb1/WvbazWa0s20+i22PsWT1zNF8+iT/5lB
H68yAQA=

```

Observation

The signed file has a different hash from the original plaintext, while the recovered plaintext hash matches the original exactly. GPG reported a good signature and identified the signing key.

Scenario Question 2.1 Answer

No, a valid digital signature does not prove that the document is factually true. It proves that the document was signed by the holder of the corresponding private key and that the content has not been changed since the signature was created. In other words, the signature proves authenticity and integrity. It does not prove that the signer was honest or that the statement inside the document is correct.

Scenario Question 2.2 Answer

Modern systems such as HTTPS use asymmetric cryptography only for the initial secure setup. The server proves its identity and the client and server establish a shared session key using public-key techniques. Once that shared key exists, the actual application data

is encrypted with fast symmetric cryptography. This hybrid design gives the convenience of secure key exchange and the speed of symmetric encryption at the same time.

Part 3: Homomorphic Encryption with TenSEAL

Exercise 3.1: Computing on Encrypted Data

Step 1: Install TenSEAL

```
~/Documents/last-term/SEN800/lab2> python3 -m pip install tenseal
Requirement already satisfied: tenseal in /opt/anaconda3/lib/python3.12/site-
packages (0.3.16)
```

Step 2: Run the original addition version

```
~/Documents/last-term/SEN800/lab2> python3 homomorphic_math.py
Vectors encrypted successfully.
Time taken for encrypted addition: 0.000442 seconds
Decrypted Result: [11, 22, 33]
Time taken for plain addition: 0.000005 seconds
Performance ratio (FHE / Plain): 88.24x slower
```

Step 3: Modify the script to perform multiplication and run it again

```
~/Documents/last-term/SEN800/lab2> python3 homomorphic_math.py
Vectors encrypted successfully.
Time taken for encrypted multiplication: 0.004198 seconds
Decrypted Result: [10, 40, 90]
Time taken for plain multiplication: 0.000005 seconds
Performance ratio (FHE / Plain): 838.43x slower
```

The first transcript above was captured before modifying `homomorphic_math.py`. The current file in the workspace is the multiplication version used for the second run.

Observation

Homomorphic computation worked correctly for both operations, but it was much slower than the equivalent plaintext computation. Multiplication was noticeably slower than addition.

Scenario Question 3.1 Answer

One strong real-world use case is healthcare analytics on outsourced infrastructure. A hospital could encrypt patient records and allow a cloud platform to compute risk scores, statistical summaries, or treatment-support values without ever seeing the raw medical data. In that environment, patient privacy, regulatory compliance, and the sensitivity of the records can matter much more than computation speed. Even if the encrypted computation is hundreds or thousands of times slower, the privacy protection can fully justify the cost.

Scenario Question 3.2 Answer

Noise is a critical limit in FHE because every operation increases the amount of random noise stored in the ciphertext. If a system performs thousands of sequential operations, the noise may grow so large that decryption can no longer recover the correct plaintext result. The computation may appear to finish, but the final decrypted value could be wrong or unreadable. That is why deep encrypted computations require careful parameter choices and remain expensive in practice.